

CODEFEST AD ASTRA 2026 — Etapa 1

Construcción de la base de conocimiento vectorial

Documento técnico de decisiones de diseño

1. Arquitectura general

El sistema es un recuperador denso multi-encoder sobre 1.826 documentos de fuentes abiertas. El flujo es: extracción de texto por formato → limpieza y normalización → fragmentación oracional con solapamiento → codificación con tres encoders independientes → un índice FAISS por encoder → fusión de rankings → dos niveles de resultado (fragmentos y documentos).

Ningún modelo generativo interviene en ninguna etapa. Ni en la construcción del índice ni en la recuperación. Todo opera sobre vectores, puntuaciones de similitud coseno y metadata, conforme a las Secciones 4.2 y 8.3.

Esa exigencia costó descartar dos cosas que funcionaban. Un baseline léxico BM25 mejoraba el F1@3 sobre el conjunto etiquetado del corpus al fusionarlo con la parte densa, y un reordenador cross-encoder, pese a no ser generativo, puntúa leyendo el par consulta-texto y no el espacio vectorial. Ninguno de los dos opera «exclusivamente sobre vectores, puntuaciones de similitud y metadata», así que ni su código ni sus pesos viajan en la entrega. El BM25 se conserva fuera de ella como instrumento de medida: es lo que permite afirmar que la parte densa aporta de verdad, y no solo que funciona.

Fenómeno	Documentos	Fragmentos aprox.
F1 — IA y capacidades estratégicas	459	—
F2 — Seguridad del entorno espacial	479	—
F3 — Dinámicas territoriales	888	—

2. Preprocesamiento

Cada archivo del corpus es un documento con un identificador único. Se usa el D0C_ID publicado por los organizadores en `Indice_Datos_Codefest.xlsx` (por ejemplo F1-AIINDEX-001); se verificó que las 1.826 entradas del inventario corresponden 1 a 1 con los archivos en disco. El nombre exacto del archivo se conserva en el campo fuente de cada fragmento, de modo que la trazabilidad al original es directa.

Formato	Documentos	Tratamiento
PDF	759	PyMuPDF en orden de lectura; OCR si no hay capa de texto
JSON	954	adaptador por observatorio; campos descriptivos a metadata
CSV	26	cada fila como pares columna: valor
XLSX	4	igual que CSV, hoja a hoja
PBF	73	atributos del tile, deduplicados entre niveles de zoom
Imagen	9	OCR (spa+eng+por)
TXT	1	texto plano

OCR. Cerca del 14% de los PDFs no tienen capa de texto. Entre ellos están los informes de seguimiento del Sistema de Alertas Tempranas de la Defensoría del Pueblo, que son material central para el Fenómeno 3: sin OCR, esos documentos serían invisibles para la recuperación. Se detecta cada página con menos de 60 caracteres extraíbles y se la procesa con Tesseract, fijando el idioma a partir de una sonda multilingüe sobre las dos primeras páginas del documento. En total se recuperaron **886 páginas** de 144 documentos, sobre 36.828 páginas de PDF procesadas.

Orden de lectura y columnas. Buena parte de los informes del corpus están maquetados a dos columnas. Extraer esas páginas ordenando los bloques por coordenada vertical —que es el comportamiento por defecto— entrelaza las dos columnas línea a línea y produce texto sin sentido: «que ningún país ha alcanzado los niveles evidenciados en países del Norte Global en el / generativa podría acelerar las tareas realizadas por los 5,69 millones de trabajadores». Para evitarlo se proyectan los bloques de cada página sobre el eje horizontal y se busca una franja vertical sin texto que separe dos grupos de masa comparable. Si existe, la página se lee columna a columna; y se divide en franjas horizontales delimitadas por los bloques que ocupan todo el ancho, de modo que un título intermedio o una tabla no rompan el orden de lectura de las columnas.

Limpieza (Sección 2.2). Normalización a UTF-8 en forma NFC, eliminación de caracteres de control, resolución de ligaduras, reunificación de palabras partidas por guion al final de línea —incluido el guion blando U+00AD, que

es marca de guionado discrecional del PDF y nunca contenido: sin tratarlo, 8.095 fragmentos de 182 documentos entraban al índice con `align ment` en lugar de `alignment`, y a diferencia del guion visible se une siempre, porque `two- and three-dimensional` sí debe conservar el suyo—, y supresión de cabeceras y pies: una línea que aparece en la banda superior o inferior de al menos el 30% de las páginas de un documento se descarta como boilerplate. El idioma predominante se detecta con un clasificador por palabras funcionales con desempate ortográfico entre español y portugués: 1022 documentos en inglés, 697 en español y 75 en portugués.

Saneamiento de títulos. El título del documento se antepone al texto de cada fragmento en el momento de codificarlo, así que un título malo contamina todos los vectores de ese documento. Los que trae la metadata de los PDFs lo son con frecuencia: nombres de archivo de InDesign o Word, la dirección postal de la imprenta, códigos internos o el encabezado de una carta. Se detectan esos casos y se sustituyen por la primera línea del cuerpo que parezca un título o, en su defecto, por el nombre del archivo humanizado —que en colecciones como el Atlas de RESDAL lleva el país del capítulo, información real y recuperable—. Quedan 26 títulos dudosos sobre 1.826 documentos.

3. Estrategia de fragmentación

Híbrida: estructural y oracional con solapamiento. El documento se parte primero por su estructura (encabezados, párrafos, filas de tabla) y un fragmento nunca cruza un encabezado. Dentro de cada sección el texto se divide en oraciones con un segmentador multilingüe (pysbd, con modelo por idioma) y las oraciones se empaquetan de forma codiciosa hasta 190 palabras, con tope duro en 240 y solapamiento de una oración entre fragmentos consecutivos.

Justificación. Tres razones concretas:

1. **Cumplimiento del requisito de completitud lingüística (Sección 3.3).** Los cortes solo ocurren en límites oracionales. Llegar ahí exigió dos correcciones no evidentes. La primera: el texto de un PDF trae un salto de línea en cada línea visual, también en mitad de una oración, y el segmentador los tomaba como frontera, partiendo frases por la mitad; los saltos internos de un bloque se convierten en espacios antes de segmentar, conservando solo los que preceden a una viñeta o a un numeral de lista. La segunda: una oración que cruza un salto de página queda repartida en dos bloques, así que si un bloque no cerró oración y el siguiente empieza en minúscula, se reúnen. El único caso en que se corta por debajo del nivel de oración es cuando una supuesta oración excede por sí sola el tope, lo que solo ocurre en filas de tabla y en texto OCR sin puntuación; ahí se cae a fronteras de cláusula, primero por punto y coma y después por coma. Afecta a 3.495 fragmentos de 91.088 (3.8%) y se contabiliza aparte.
2. **Los bloques que no son prosa se descartan.** Índices de contenidos con puntos suspensivos, listas de figuras y bloques de cifras puras no aportan significado recuperable, y al competir por un puesto en el top-10 desplazan fragmentos que sí responden a la consulta.
3. **El tope de 240 palabras está por debajo del límite de 250 de la salida.** Así ningún fragmento recuperado necesita subdividirse al entregarlo (Sección 9.2.1), lo que elimina de raíz una fuente entera de errores de formato. Deja además margen para enriquecer el fragmento con contexto vecino, como se explica en la Sección 7.
4. **El buffer persiste entre párrafos de una misma sección.** Un párrafo suelto de veinte palabras —una línea de autores, un pie de página, un encabezado de capítulo— no debe convertirse en un fragmento propio: sería ruido compitiendo por un lugar en el top-10. Al empaquetar a través de párrafos, la mediana subió de 96 a 198 palabras y el número de fragmentos bajó un 25%.

Resultado: **91.088 fragmentos**, media 185 palabras, mediana 198, percentil 90 224, máximo 240. Ningún documento se queda sin fragmentos: los archivos sin

texto extraíble reciben un fragmento de respaldo con su título y procedencia, para que su `doc_id` siga siendo recuperable.

Metadata. Cada fragmento lleva los ocho campos obligatorios de la Tabla 2 (`doc_id`, `chunk_id`, `fuentes`, `formato`, `fenomeno`, `posicion`, `num_tokens`, `texto`) más campos adicionales que la recuperación aprovecha: `observatorio`, `título`, `idioma`, `fecha`, `url`, `seccion` y `ruta_relativa`. El campo `num_tokens` se calcula con el tokenizador de cada encoder, por lo que cada metadata.jsonl reporta el conteo que le corresponde.

Para codificar se usa `título | sección | texto`, mientras que el campo `texto` que se almacena y se entrega es el fragmento literal, sin modificaciones. Esto da al vector el contexto del documento sin alterar el texto evaluado.

4. Selección de encoders

Se emplean tres encoders complementarios, cada uno con su propio índice FAISS (Sección 4.4).

Encoder	Arquitectura	Licencia	Dim.	Índice
granite-embedding-311m-multilingual-r2	ModernBERT (encoder-only)	Apache 2.0	768	280 MB
bge-m3	XLNet-RoBERTa (encoder)	MIT	1024	373 MB

multilingual-e5-large	XLNet-RoBERTa (encoder)	MIT	1024	373 MB
-----------------------	-------------------------	-----	------	--------

Los tres son arquitectura encoder. Es la restricción que más condiciona la elección: varios de los modelos mejor situados hoy en MTEB multilingüe (Qwen3-Embedding, gte-Qwen2, e5-mistral) están contruidos sobre un backbone decoder, y la Sección 4.2 los excluye explícitamente. Los tres seleccionados son bidireccionales de la familia BERT.

Justificación contra los seis criterios de la Sección 4.3:

- **Soporte multilingüe.** El corpus mezcla español, inglés y portugués, y las consultas vienen en los tres idiomas: hace falta que una consulta en español recupere documentos en inglés. Los tres modelos operan nativamente en los tres idiomas y comparten espacio semántico entre ellos.
- **Dimensionalidad.** 768 y 1024. Más dimensiones no garantizan mejor rendimiento; se prefirió diversidad de espacios (un modelo de 768 y dos de 1024, con backbones distintos) sobre dimensionalidad bruta, porque la ganancia de la fusión viene de que los errores de cada modelo no estén correlacionados.
- **Longitud máxima de entrada.** Los tres admiten al menos 512 tokens; la fragmentación se diseñó para no acercarse a ese límite.
- **Rendimiento en benchmarks.** Los tres tienen buen desempeño documentado en recuperación densa multilingüe (MTEB y MIRACL), no en tareas de clasificación o similitud de pares.
- **Licencia.** Apache 2.0 y MIT, ambas compatibles con el uso en el reto.
- **Eficiencia.** Entre 311M y 568M parámetros. La codificación del corpus completo se resolvió en horas en una sola máquina, con inferencia en media precisión: la diferencia respecto de precisión simple es de $2 \cdot 10^{-4}$ en similitud coseno, irrelevante para el orden del ranking, a cambio de casi el triple de velocidad.

5. Índice vectorial

IndexFlatIP sobre vectores normalizados a norma unitaria. Con la norma fijada a uno, el producto interno equivale exactamente a la similitud coseno (Sección 8.2, ecuación 4), de modo que el índice devuelve el resultado exacto sin aproximación. Con 91.088 vectores la búsqueda exhaustiva tarda milisegundos, así que un índice aproximado (IVF o HNSW) solo aportaría pérdida de exactitud a cambio de una velocidad que no necesitamos.

Cada encoder tiene su carpeta con `index.faiss` (serializado con `faiss.write_index`, cargable con `faiss.read_index` sin dependencias adicionales) y su `metadata.jsonl`, escrito en el mismo orden en que los vectores se insertaron en el índice. Como IndexFlatIP asigna identificadores internos secuencialmente, la línea i del `metadata` corresponde al vector i . La verificación reconstruye vectores del índice y los compara con los originales.

6. Grafo de conocimiento (componente bonus)

El grafo tiene **33.178 entidades** y **331.721 relaciones**, con trazabilidad completa: cada arista guarda los `chunk_id` y `doc_id` donde se observó la relación, como pide la Sección 7.2.

Reconocimiento de entidades. Tres fuentes complementarias. Primero, NER estadístico con spaCy en `_core_web_md` (MIT) sobre los fragmentos en inglés, que son el 76,5% del corpus. Segundo, un extractor de nombres propios por reglas —secuencias capitalizadas con sus partículas, desambiguadas por posición en la oración y tipadas por pistas léxicas— para español y portugués. Esa asimetría no es técnica sino de licencia, y se explica en la Sección 8. Tercero, gazetteers específicos del dominio, porque ningún NER genérico reconoce justamente las entidades que estas consultas necesitan. Los gazetteers territoriales no están escritos a mano: se derivan de los datos estructurados que el propio corpus trae —el campo `municipios` de las alertas tempranas de la Defensoría y las columnas administrativas del conjunto de Amazon Underworld—, y se complementan con listas curadas de grupos armados, economías ilícitas, tecnología de IA militar, vocabulario espacial y marcos jurídicos.

Sobre las tres fuentes actúa un filtro de artefactos tabulares. Los CSV masivos del AI Index se aplanan a `columna:valor` con `|` entre filas, y cualquier NER lee esas cadenas como nombres propios: `Journal/Book: J Urol, Li J. |`, encabezados de capítulo, URLs. Eran 547 nodos, siete de ellos entre los cien más conectados. El filtro apunta a esas formas concretas y no a los dos puntos en general, porque hay entidades legítimas que los llevan: ISO 24113:2023 es la norma de mitigación de residuos orbitales y es material central del Fenómeno 2.

Tipos de entidad: `organizacion` (16248), `persona` (6681), `otro` (5480), `lugar` (2907), `marco_juridico` (831), `municipio` (654).

Extracción de relaciones. Donde hay árbol de dependencias —los fragmentos en inglés— se recorre cada verbo, se toman los subárboles de sus hijos con función de sujeto y de objeto, y se buscan entidades conocidas dentro de cada uno. El lema del verbo es el tipo de relación. Las relaciones más frecuentes son `announced` (610), `create` (468), `stated` (413), `posted` (397), `said` (265). Los pares de entidades sin conexión sintáctica, y todos los pares de los fragmentos procesados por reglas, se registran como `co_ocurre_con`, que se poda si no aparece al menos dos veces.

Integración con la recuperación (Sección 8.5). Las entidades de la consulta se detectan con los gazetteers, no con el NER estadístico: son código propio y tabla de alias, así que la consulta se resuelve sin cargar ningún modelo de

lenguaje. Desde ellas se recuperan los fragmentos vinculados a esas entidades y a sus vecinos de primer orden en el grafo, y se puntúan por el número de relaciones relevantes encontradas. La lista resultante entra al RRF como un índice más.

7. Módulo de recuperación

La consulta se codifica con los mismos tres encoders usados en la indexación, cada índice devuelve sus 200 candidatos más similares, y las listas se combinan. Después:

Fusión (Sección 8.4). Se usa **Reciprocal Rank Fusion** con $k = 60$. Frente a CombSUM y CombMNZ, RRF combina rangos en lugar de puntuaciones, lo que lo hace robusto a que los tres encoders produzcan distribuciones de similitud con escalas distintas —que es exactamente nuestro caso—. Las tres alternativas se compararon empíricamente (Sección 8).

Prior temático suave (Sección 8.7). Se calcula el centroide de cada fenómeno como la media de los vectores de sus fragmentos, y los fragmentos del fenómeno más cercano a la consulta reciben un impulso multiplicativo pequeño. Es deliberadamente un impulso y no un filtro: hay consultas que cruzan fenómenos —preguntar por inteligencia artificial en operaciones espaciales toca F1 y F2 a la vez— y un filtro duro perdería documentos relevantes.

Tope de diversidad. Como máximo 3 fragmentos por documento en la lista de diez. Sin ese tope, un documento largo y muy pertinente puede copar la lista entera y desperdiciar posiciones que otro documento relevante aprovecharía.

Degradación de los índices de contenidos. Un índice de PDF —la tabla de títulos unidos a su número de página por una fila de puntos— contiene todos los términos de la consulta y ninguna respuesta. La similitud lo premia por eso mismo, y el NDCG lo castiga: en una de las cincuenta consultas uno de ellos ocupaba la primera posición, la de mayor peso de la métrica. Se reconocen por los líderes de puntos, dos o más por fragmento, y en el corpus lo cumplen 1.115 de los 91.088 fragmentos, todos ellos índices reales. No se descartan, porque el enunciado permite conservarlos como parte del documento: se mandan al final de la lista de candidatos, de modo que solo aparecen si no hubiera diez fragmentos mejores.

Agregación a documento (Sección 8.6). La puntuación de un documento es la de su mejor fragmento, sin más. La primera versión sumaba una contribución decreciente de los siguientes fragmentos, con la idea de premiar a los documentos que tratan el tema en varios pasajes. Resultó ser un error: las puntuaciones RRF decaen muy poco a lo largo del ranking —0,0164 en el puesto 1 frente a 0,0091 en el 50—, de modo que un documento con cinco fragmentos mediocres superaba a uno con un único fragmento excelente. Quitar esa cola subió la sonda de títulos de 0,45 a 0,92 y la de known-item de 0,78 a 0,92 (Sección 8). F1@3 es una métrica de conjunto, así que se optimiza el acierto y no el orden.

Supresión de documentos duplicados. El corpus contiene el mismo documento bajo nombres distintos —ILIA_documento-ilia-2025 e ILIA_documento-ilia-web tienen coseno 1,000— y ediciones traducidas del mismo informe. Sin tratarlo, 18 de las 50 consultas gastaban dos de sus tres posiciones en el mismo contenido. Se comparan los centroides de documento en los tres encoders, exigiendo que los tres coincidan, porque la densidad del espacio varía mucho entre modelos: con umbral 0,98 granite marca 1.750 pares casi idénticos entre 1.826 documentos y bge-m3 solo 175.

Un único umbral no basta. Una traducción del mismo informe y una edición posterior dan ambas coseno 0,956, y confundirlas es caro: el conjunto de validación marca los informes semestrales 35 y 36 de la MAPP/OEA como relevantes a la vez para una misma pregunta. El idioma, que viaja en la metadata, sí las distingue, de modo que se descarta un documento si es copia literal (coseno > 0,98) o si es una traducción (coseno > 0,95 y en otro idioma). Una edición posterior en el mismo idioma se conserva.

Expansión con contexto vecino (Sección 9.2.1). La fragmentación deja el 80% de los fragmentos por encima de las 180 palabras, así que la mayoría se entrega tal cual. Para el resto —fragmentos cortos, típicamente el cierre de una sección— se rellena el presupuesto de 250 palabras con los vecinos inmediatos del mismo documento, alternando posterior y anterior para que el fragmento recuperado quede centrado, eliminando la oración de solapamiento para no duplicarla, y tomando del vecino solo oraciones completas cuando no cabe entero. El chunk_id reportado sigue siendo el del fragmento original del índice, que cumple función de trazabilidad. La especificación lo permite explícitamente y aumenta la probabilidad de que el fragmento entregado contenga el pasaje relevante completo.

8. Evaluación interna y reproducibilidad

El ground truth no es público, así que la configuración se ajustó con cuatro señales independientes: las preguntas etiquetadas que venían dentro del corpus; una sonda *known-item* que usa fragmentos del propio corpus como pseudo-consultas y mide si vuelve su documento de origen; una sonda que consulta con el título de cada documento; y revisión manual de las cincuenta consultas.

Configuración	F1@3	Cobertura	Known-item	Títulos	Duplic.
---------------	------	-----------	------------	---------	---------

solo granite	0,042	0,450	0,808	0,842	0,04
solo bge_m3	0,175	0,444	0,850	0,917	0,28
solo e5_large	0,050	0,319	0,800	0,925	0,08
fusión rrf	0,092	0,478	0,850	0,917	0,30
fusión combsum	0,092	0,454	0,850	0,917	0,32
fusión combmnz	0,092	0,435	0,850	0,917	0,32
rrf + prior fenómeno	0,092	0,478	0,850	0,917	0,30
rrf sin expansión de vecinos	0,092	0,471	0,850	0,917	0,30
rrf, expansión a 200 palabras	0,092	0,474	0,850	0,917	0,30
rrf, expansión a 225 palabras	0,092	0,475	0,850	0,917	0,30
rrf, expansión a 245 palabras	0,092	0,478	0,850	0,917	0,30
rrf, expansión a 250 palabras	0,092	0,479	0,850	0,917	0,30
sin poda de duplicados	0,092	0,478	0,850	0,917	0,36
poda duplicados cos>0.98	0,092	0,478	0,850	0,917	0,30
poda duplicados cos>0.95	0,092	0,478	0,850	0,917	0,28
poda duplicados cos>0.92	0,092	0,478	0,850	0,917	0,00
rrf, cola doc peso 0.0 n=0	0,092	0,478	0,850	0,917	0,30
rrf, cola doc peso 0.05 n=4	0,092	0,478	0,850	0,917	0,38
rrf, cola doc peso 0.15 n=4	0,092	0,478	0,850	0,917	0,38
rrf, cola doc peso 0.35 n=4	0,042	0,478	0,850	0,917	0,36
rrf, máx 2 frag/doc	0,092	0,478	0,850	0,917	0,30
rrf, máx 3 frag/doc	0,092	0,478	0,850	0,917	0,30
rrf, máx 4 frag/doc	0,092	0,478	0,850	0,917	0,30
rrf, máx 10 frag/doc	0,092	0,478	0,850	0,917	0,30
rrf + grafo	0,092	0,478	0,833	0,917	0,30
rrf + grafo + prior	0,092	0,478	0,833	0,917	0,30

La columna **Duplic.** es la fracción de las cincuenta consultas reales cuyo top-3 contiene dos documentos con contenido casi idéntico. Se incluye porque F1@3 sobre ocho preguntas etiquetadas no tiene resolución para distinguir configuraciones —casi toda la tabla queda plana— mientras que esa medida sí separa las decisiones y se calcula sobre las consultas que de verdad se evalúan. Las filas de un solo encoder no son comparables con las de fusión en las sondas, porque la supresión de duplicados exige acuerdo entre los encoders disponibles y con uno solo se vuelve más agresiva.

Reproducibilidad. `generador.py` se ejecuta sin argumentos desde la raíz de la entrega y regenera `resultados.jsonl`. El código de la librería viaja dentro de la entrega en `lib/`, las versiones de las dependencias están fijadas en `requirements.txt`, y el lector de consultas acepta las variantes razonables de nombre de campo para que una diferencia de nomenclatura no impida la ejecución. Al terminar, el propio script valida su salida contra el esquema de la Sección 9.3 y devuelve código de error si algo no cumple.

La comprobación no se dio por buena sobre el papel: se creó un entorno virtual limpio, se instaló únicamente lo que declara `requirements.txt`, se copió la entrega fuera del árbol de desarrollo y se ejecutó `python generador.py` sin argumentos. La salida se reprodujo byte a byte.

Esa prueba destapó dos fallos, y los dos tenían la misma forma: la entrega se comportaba distinto según el procesador de la máquina, y la propia prueba no podía verlo porque corría siempre en la misma.

El primero. `faiss-cpu` y `torch` empaquetan cada uno su copia de `libomp`, y cargar las dos en un proceso aborta el intérprete con `OMP: Error #15. La variable KMP_DUPLICATE_LIB_OK silencia ese mensaje, pero solo el mensaje: los dos runtimes siguen levantando su propio pool de hilos y el segundo mata el proceso con SIGSEGV`. En un equipo con GPU no se ve, porque `torch` trabaja en `mps` o `cuda` y nunca llega a levantar el pool; en CPU sí, que es justo donde corre quien evalúa, porque `requirements.txt` fija `faiss-cpu`. Ni la variable de entorno `OMP_NUM_THREADS` lo evita:

torch fija sus hilos al inicializar su runtime, después de leerla. Hay que decírselo por API, y el arreglo es dejar a faiss y a torch con un hilo cada uno antes de tocar ninguno de los dos.

El segundo, sentence-transformers usa media precisión en mps y cuda, y fp32 en CPU, de modo que el vector de una misma consulta dependía de la máquina. No es un detalle numérico sin consecuencias: sobre las cincuenta consultas reales, codificar en CPU en vez de en mps cambiaba la lista de fragmentos de quince de ellas y el top-3 de documentos de una. El resultados.jsonl entregado no habría sido el que reproduce quien evalúa. La consulta se codifica ahora siempre en CPU y en fp32, que es el único punto de operación que cualquier máquina reproduce, y no cuesta calidad: con las sondas de conocido y de títulos sobre doscientas pruebas, las cuatro señales se mantienen o mejoran. Entre arquitecturas quedan diferencias de orden $1e-7$, cuatro órdenes de magnitud por debajo de las que separan media precisión de fp32, y muy por debajo de lo que mueve un ranking.

La equivalencia numérica de la búsqueda se verificó además contra NumPy puro sobre los 91.088 vectores: los identificadores recuperados coinciden salvo empates con diferencia de puntuación menor que $1e-4$, y las puntuaciones ordenadas coinciden a $1e-5$.

Licencias de todos los componentes. No se dieron por buenas las licencias que figuran en la documentación de cada proyecto: se leyó la que declara cada artefacto instalado. Los tres encoders son Apache 2.0 y MIT, y faiss, spaCy, blingfire y NetworkX son MIT o BSD.

La verificación cambió una decisión de diseño. Los modelos de NER de spaCy declaran en su propio meta.json licencias distintas entre sí: en_core_web_md es MIT, pero es_core_news_md es GNU GPL 3.0 y pt_core_news_md es CC BY-SA 4.0. Ninguna de las dos últimas encaja en las licencias que la Sección 4.3 señala como preferentes, y la segunda es la licencia sobre la que los organizadores dictaminaron expresamente. Las alternativas multilingües revisadas no mejoran el cuadro: wikineural-multilingual-ner es CC BY-NC-SA 4.0 y la familia xlm-roberta-*-ner-hrl es AFL-3.0. Por eso el grafo se reconstruyó usando el modelo estadístico solo en inglés —que es el 76,5% del corpus— y cubriendo español y portugués con el extractor por reglas descrito en la Sección 6, que al ser código propio no arrastra licencia de terceros. Se prefirió un grafo algo más pobre y enteramente permisivo a uno más rico y discutible.

Conviene decir qué costó: el grafo pasó de 35.022 a 33.178 entidades y de 346.487 a 331.721 relaciones. Parte de esa diferencia no es pérdida sino limpieza: al revisar el grafo aparecieron 547 nodos que no eran entidades sino artefactos del aplanado de tablas —cabeceras de columna de los CSV del AI Index, fragmentos de listas de autores, URLs— y se filtraron. El efecto sobre la recuperación queda dentro del ruido de medida, lo que era esperable: el grafo aporta una lista más al RRF y su contribución ya era marginal frente a los tres índices densos.